

第 45 讲：为什么需要 Normalization

网络越深，Activation 的尺度为什么越来越难控制？

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【科研洞察】

第 40 讲已经看到：即使 Initialization 选得不错，深层网络中的数值尺度仍然可能在层与层之间不断变化。第 43-44 讲又看到：Regularization 解决的是泛化问题，而不是所有训练稳定性问题。**Normalization 的核心任务之一，就是让网络中的数值尺度更可控，让后续层面对的输入不至于一会儿极小、一会儿极大。**

本讲先回答“为什么需要 Normalization”，暂时不把 BatchNorm / LayerNorm / RMSNorm 的细节一次讲完。第 46、47 讲会分别深入这些具体方法。

一、先把问题说清楚：Normalization 到底在“规范”什么？

一个 Tensor 不只有 Shape，还有**数值分布**。最常用的两个尺度统计量是：

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

其中：

- μ ：均值（mean），描述整体中心在哪；
- σ^2 ：方差（variance），描述数字围绕均值散得多开；
- $\sigma = \sqrt{\sigma^2}$ ：标准差（standard deviation），和原数据单位一致，更直观。

【原理】

Normalization 并不是“把所有数字都变成一样”，而是按照某种统计规则，把一组数字重新放到一个更可控的尺度上。最经典的标准化形式是：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

二、【具体数字例子】先手算一次标准化

取：

$$x = [2, 4, 6, 8]$$

第一步，均值：

$$\mu = \frac{2 + 4 + 6 + 8}{4} = 5$$

第二步，每个数减均值：

$$x - \mu = [-3, -1, 1, 3]$$

第三步，总体方差：

$$\sigma^2 = \frac{9 + 1 + 1 + 9}{4} = 5$$

第四步，标准差：

$$\sigma = \sqrt{5} \approx 2.236$$

第五步，标准化：

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

此时：

$$\text{mean}(\hat{x}) \approx 0, \quad \text{std}(\hat{x}) \approx 1$$

【具体例子】

如果把原数据整体放大 10 倍再加 100：

$$x' = [120, 140, 160, 180]$$

重新标准化后，仍然几乎得到：

$$[-1.342, -0.447, 0.447, 1.342]$$

因为“整体平移”和“整体正比例放大”被减均值、除标准差消掉了。

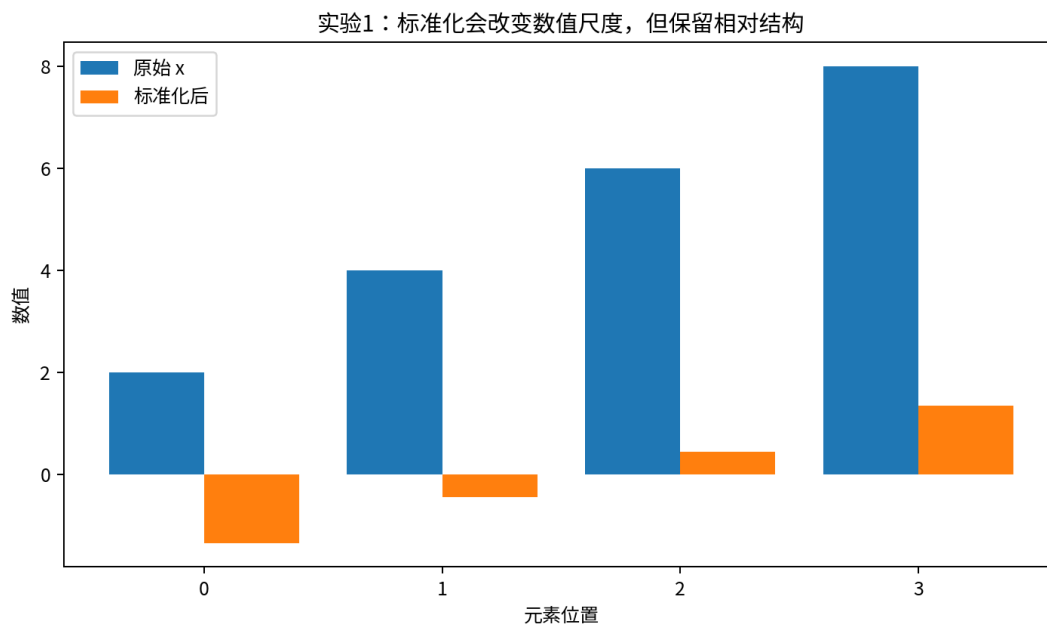


图 1：实验 1：标准化改变绝对尺度，但保留一组数字内部的相对位置结构。

三、为什么深度学习特别在意“尺度”？

深度学习里，一个数不会只经过一次运算，而会经历很多层：

$$X \rightarrow \text{Linear} \rightarrow \text{Activation} \rightarrow \text{Linear} \rightarrow \text{Activation} \rightarrow \dots$$

每一层都可能：

- 乘 Weight；
- 加 Bias；
- 经过非线性；
- 和其他分支相加；
- 再作为下一层输入。

所以某一层的均值和方差发生变化，下一层接收到的“工作环境”就会跟着变化。

1. 太小：信号可能逐渐消失

如果一层又一层把数值压小，Activation 可能逐渐接近 0；Gradient 也可能在 Chain Rule 中逐层变小。

2. 太大：激活与梯度可能失控

如果数值不断被放大，可能导致：

- Activation 爆大；
- Loss 变大；
- Gradient 爆炸；
- 数值溢出；
- Optimizer 很难使用一个合适的全局 Learning Rate。

3. 不同维度差异太大：同一个 LR 很难同时照顾

这是一个非常重要但容易忽略的问题。

假设：

$$x = [1, 100, 10000]$$

最简单线性模型：

$$\hat{y} = w^T x$$

Loss：

$$L = (\hat{y} - y)^2$$

当当前 $\hat{y} = 0, y = 1$ 时:

$$\frac{\partial L}{\partial w_i} = 2(\hat{y} - y)x_i = -2x_i$$

于是:

$$\frac{\partial L}{\partial w} = [-2, -200, -20000]$$

三个参数对应的梯度尺度直接相差 10^4 倍。

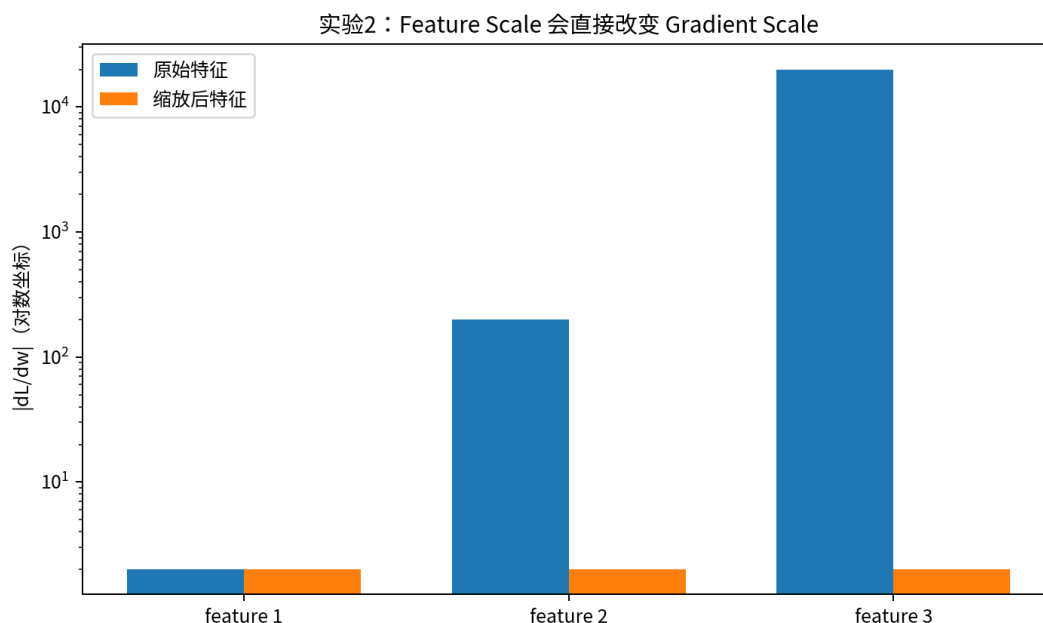


图 2: 实验 2: 输入特征尺度差异会直接传递到 Weight Gradient 的尺度。

【科研洞察】

这就是为什么“数值尺度”不仅是好不好看的问题，而会直接影响 Optimization Geometry: 某些方向很陡，某些方向很平，一个全局 Learning Rate 很难同时兼顾。

四、深层网络里的“尺度漂移”是什么样子？

本讲实验构造一个 20 层 ReLU 网络。为了让现象容易观察，每层 Weight 比标准 He 尺度略大，并带小正 Bias。

无 Normalization 时，Activation Std:

$$1.006 \rightarrow 1.972 \rightarrow 3.711 \rightarrow 17.445$$

同时 Activation Mean:

$$0.703 \rightarrow 1.210 \rightarrow 2.234 \rightarrow 9.623$$

也就是说：越往后，下一层面对的输入数值环境越来越不同。

如果在每层 ReLU 前，先做“概念性的按最后一个特征维标准化”，Activation Std 则大致保持:

$$0.58, 0.57, 0.61, 0.57$$

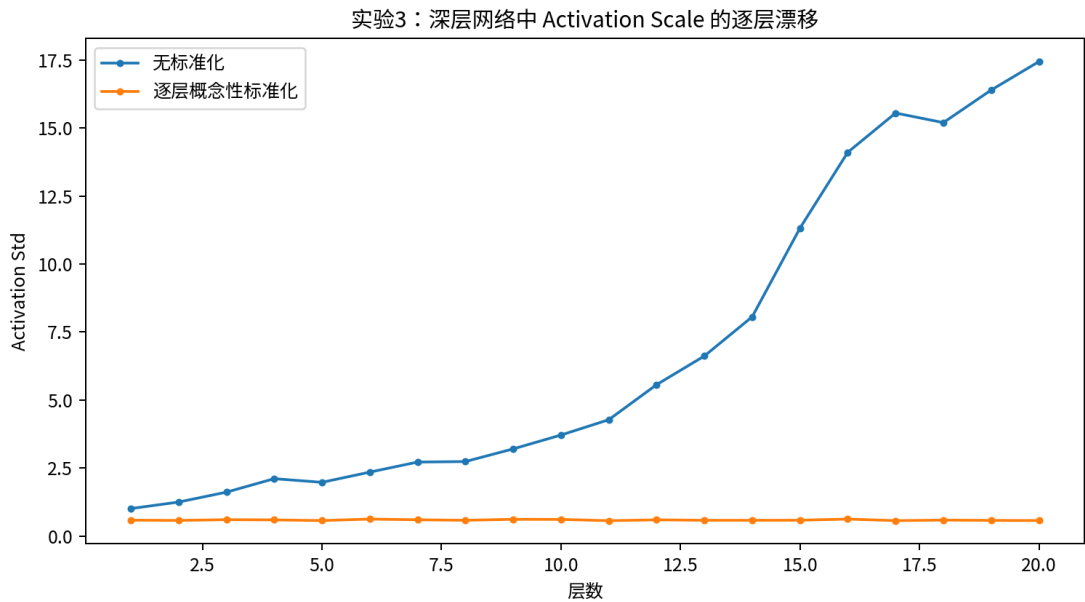


图 3: 实验 3: 同一组 Weight 下, 概念性标准化显著抑制深层 Activation Scale 漂移。

附近。

【常见误区 / 风险】

这里使用的是“概念性标准化”来展示原理, 不是说真实模型一定要在每一层用同一种标准化方式。BatchNorm、LayerNorm、RMSNorm 的关键区别之一, 恰恰就在“到底对哪些元素计算统计量”。

五、Normalization 为什么会影响 Activation Function ?

回忆第 9 讲: Activation Function 并不是在任何输入范围都表现一样。

以 Sigmoid 为例:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

导数:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

当 $|z|$ 很大时, Sigmoid 输出接近 0 或 1, 导数接近 0, 这就是**饱和 (saturation)**。

Tanh 同样存在饱和:

$$\tanh'(z) = 1 - \tanh^2(z)$$

本讲实验统计 “导数 < 0.01 ” 的比例。输入标准差越大, 进入饱和区的样本越多:

输入 Std	0.5	1	2	5	10
Sigmoid 饱和比例	0	0	0.022	0.359	0.644
Tanh 饱和比例	0	0.003	0.135	0.550	0.762

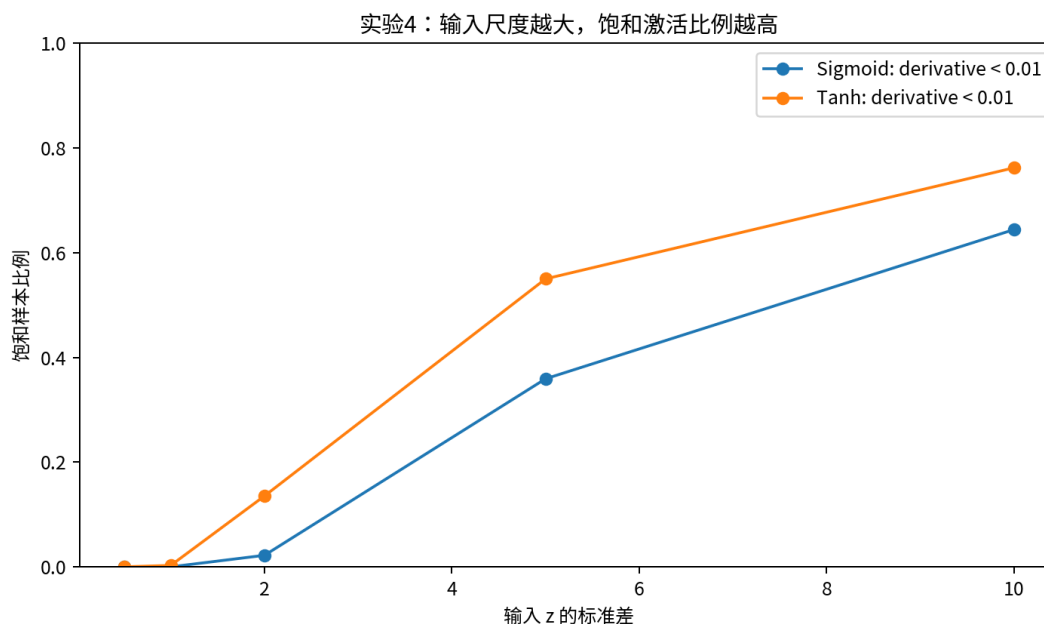


图 4: 实验 4: 输入尺度变大, 会显著提高 Sigmoid / Tanh 进入低梯度饱和区的概率。

【原理】

Normalization 不能保证梯度永不消失, 但它可以帮助把中间激活保持在更可控的数值范围, 减少网络因为尺度漂移而频繁进入极端区域。

六、为什么不能只靠 Initialization ?

第 40 讲已经学过 Xavier / He Initialization。它们解决的是：

第 0 Step 开始时, 尽量给网络一个合理的信号传播条件。

但训练开始以后：

- Weight 会被 Optimizer 不断更新；
- 每层输入分布会随着前面层改变；
- 数据 Batch 也在变化；
- Residual、Attention、MLP 等不同分支会组合；
- 激活统计量不可能永远等于初始化时的状态。

所以 Initialization 和 Normalization 不是二选一：

Initialization “ ” Normalization “ ”

七、标准化以后为什么还需要 γ 和 β ?

如果所有输出永远被强制为均值 0、标准差 1, 会不会限制网络表达能力?

因此很多 Normalization Layer 会在标准化后再做：

$$y = \gamma \hat{x} + \beta$$

其中：

- γ : learnable scale;
- β : learnable shift;
- $\hat{x}$: 标准化后的数值。

也就是说，Normalization 并不是要永久锁死模型的均值和尺度，而是先把输入变成稳定基准，再让模型通过可学习参数重新选择合适的尺度与偏移。

【具体例子】

如果标准化后：

$$\hat{x} = [-1, 0, 1]$$

取：

$$\gamma = 2, \quad \beta = 3$$

那么：

$$y = 2\hat{x} + 3 = [1, 3, 5]$$

因此网络仍然具有重新缩放和移动表示的能力。

八、 ϵ 为什么必须存在？

标准化分母是：

$$\sqrt{\sigma^2 + \epsilon}$$

如果一组输入完全相同：

$$[5, 5, 5, 5]$$

则：

$$\sigma^2 = 0$$

如果没有 ϵ ，就会发生除以 0。

ϵ 通常是很小的正数，例如 10^{-5} 一类量级。它的作用是数值稳定性保护。

【常见误区 / 风险】

加入 ϵ 后，标准化结果的方差通常“接近 1”而不是数学上严格等于 1；特别是原始方差很小时， ϵ 的影响会更加明显。

九、最关键的问题：到底对哪些元素求 Mean / Variance？

这是接下来两讲的核心。

假设 Transformer hidden states:

$$X.shape = (B, T, D)$$

例如:

$$(32, 128, 768)$$

“做 Normalization” 这句话仍然不完整，因为必须继续问：

Mean 和 Variance 到底沿哪个 Axis 计算？

不同方法的根本区别之一就在这里：

- BatchNorm: 统计量依赖 Batch / 空间维度的组织方式；
- LayerNorm: 通常对单个样本的特征维做归一化；
- RMSNorm: 不显式减去 mean，主要控制 RMS 尺度。

本讲先不要求背这些细节，只要求建立一个科研阅读习惯：

【科研洞察】

任何看到 “Normalization” 时，第一反应不应该只是 “把数变成均值 0 方差 1”，而应该追问：**统计量是对哪些元素算的？训练和推理用的是不是同一套统计量？有没有 learnable scale / bias？**

十、PyTorch 预览：LLM 风格 (B, T, D) Tensor

为了提前建立直觉，本讲只做一个 LayerNorm 功能预览，详细原理放到第 47 讲。

输入：

$$X.shape = (2, 2, 4)$$

某两个 token 分别是：

$$[1, 2, 3, 4]$$

和：

$$[10, 20, 30, 40]$$

后者只是前者整体放大 10 倍。

执行：

```
1 import torch
2 import torch.nn as nn
3
4 # 对最后一个 hidden 维做归一化（这里只做功能预览）
5 ln = nn.LayerNorm(4, elementwise_affine=False)
6
7 X = torch.tensor([
8     [[1., 2., 3., 4.],
9      [10., 20., 30., 40.]]
```

```

10 ]])
11
12 Y = ln(X)
13 print(Y.shape)

```

Shape 保持:

$$(1, 2, 4) \rightarrow (1, 2, 4)$$

两个 token 归一化以后几乎得到完全相同的形状:

$$[-1.342, -0.447, 0.447, 1.342]$$

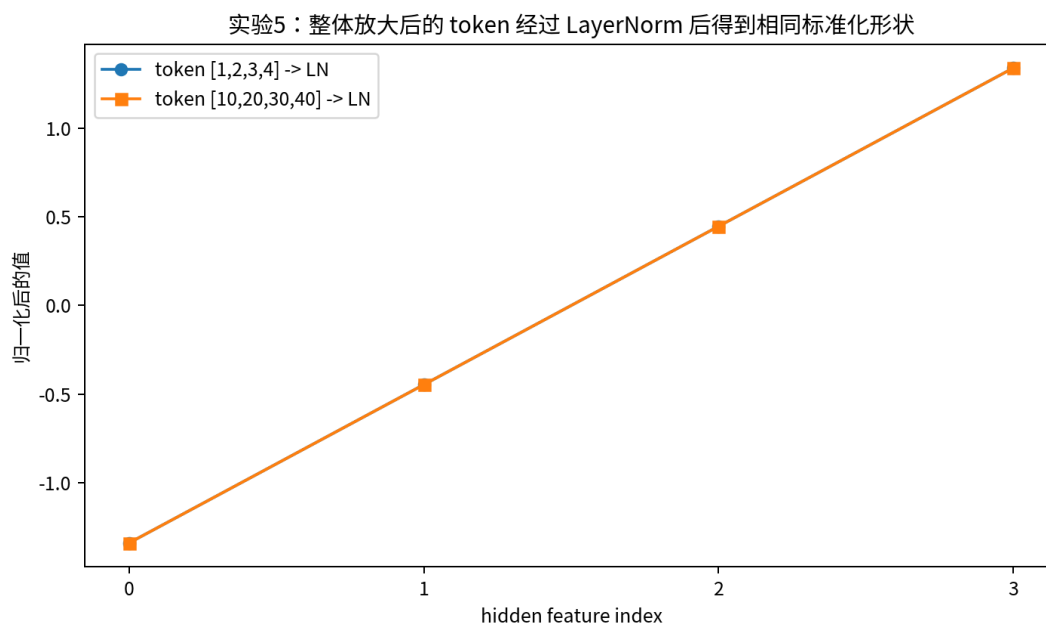


图 5: 实验 5: 整体正比例放大的两个 token, 经 LayerNorm 后得到近乎相同的标准化形状。

十一、Normalization 与 Data Preprocessing 是一回事吗？

不是。

输入数据预处理

例如表格特征标准化:

$$x'_j = \frac{x_j - \mu_j^{train}}{\sigma_j^{train}}$$

统计量通常应该只用 Training Set 计算, 再应用到 Validation / Test, 防止 Data Leakage。

网络内部 Normalization Layer

它发生在模型 Forward 内部, 处理的是中间 Activation / Hidden State, 并且可能带 learnable Parameter、running statistics 或不同 axis 规则。

【常见误区 / 风险】

如果你在数据预处理阶段用 “Train + Validation + Test 全部数据” 一起计算均值和标准差，就已经发生 Preprocessing Leakage。这个问题属于第 42 讲的 Data Leakage，而不是 Normalization Layer 本身。

十二、Normalization 与 Regularization 是一回事吗？

也不是。

Weight Decay、Dropout 的主要课程定位是：

限制过拟合 / 改善 Generalization

Normalization 的主要课程定位是：

控制中间表示尺度 / 改善 Optimization 与训练稳定性

现实中某些 Normalization 方法也可能带来正则化效应，但不能把二者直接画等号。

十三、为什么 Transformer 更关心 LayerNorm / RMSNorm？

在 CNN 时代，BatchNorm 非常常见；进入 Transformer / LLM 后，LayerNorm、RMSNorm 更常见。

先给出直觉，不在本讲展开证明：

- 文本序列长度可能变化；
- Batch Size 可能因显存而变化；
- 一个 token 的 hidden vector (D ,) 本身就是很自然的归一化单元；
- LLM 推理时经常是很小 Batch，甚至单样本；
- Transformer Block 的 Residual + Attention + MLP 需要稳定 hidden scale。

因此以后你会频繁看到：

$$\text{LayerNorm}(X)$$

或：

$$\text{RMSNorm}(X)$$

嵌在 Transformer Block 中。

【科研洞察】

你以前在 LLM 课程里见过 Transformer Block 里的 LayerNorm / RMSNorm；现在我们从底层重新理解：它们首先是在解决**深层 hidden state 的数值尺度与优化稳定性问题**，而不只是架构图上的一个固定方框。

十四、Normalization 这个词很容易和哪些东西混淆？

“Normalization / 归一化”在不同论文和工程代码中并不总是指同一件数学操作。零基础阶段先把几个常见概念分开。

1. Standardization：减均值、除标准差

本讲主要讨论的基本形式：

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

目标是控制中心和尺度。

2. Min-Max Scaling：压到固定区间

例如：

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

常把数据压到 $[0, 1]$ 。它和“均值 0、标准差 1”不是一回事。

3. L2 Vector Normalization：把向量长度变成 1

例如：

$$\hat{x} = \frac{x}{\|x\|_2}$$

常见于 cosine similarity / embedding 检索。它控制的是向量范数，不一定让 mean=0 或 std=1。

4. Softmax Normalization：把一组 score 变成概率分布

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

它满足 $\sum_i p_i = 1$ ，但这和 LayerNorm / BatchNorm 的统计标准化完全不同。

【常见误区 / 风险】

以后看到论文写 “normalize”，一定要看公式或代码。只看这个单词本身，无法判断作者到底做的是 standardization、unit-norm、probability normalization，还是别的尺度处理。

十五、Axis 为什么是 Normalization 中最核心的问题之一？

先看一个二维矩阵：

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 10 & 20 & 30 \end{bmatrix}$$

同一个 X ，可以有完全不同的归一化方式。

按行 (row) 标准化

第一行自己算 mean/std, 第二行自己算 mean/std。结果两行都会变成相似标准形状:

$$[-1.225, 0, 1.225]$$

这强调 “每个样本内部的 feature 相对关系”。

按列 (column) 标准化

第一列 [1, 10] 一组, 第二列 [2, 20] 一组, 第三列 [3, 30] 一组。这强调 “同一个 feature 在不同样本之间的统计关系”。

【原理】

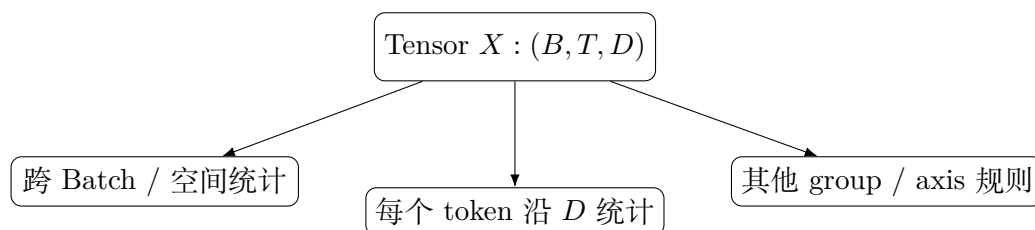
Normalization 的公式可能完全一样, 但只要计算 μ, σ^2 的 Axis 不同, 模型行为就可能完全不同。

对于 Transformer:

$$X.shape = (B, T, D)$$

我们以后至少要能明确区分:

- 跨 Batch 统计;
- 跨 Token 统计;
- 对每个 token 的 Hidden Dimension D 统计;
- 是否把多个维度合并为一个统计组。



十六、Normalization 的 Backward 为什么不是简单逐元素?

ReLU 是逐元素的: 一个输出 y_i 只依赖一个输入 x_i 。但标准化不同。

因为:

$$\mu = \frac{1}{N} \sum_j x_j$$

以及:

$$\sigma^2 = \frac{1}{N} \sum_j (x_j - \mu)^2$$

所以某个:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

不仅依赖 x_i ，还依赖同一 normalization group 中其他 x_j 。

因此，Backward 时：

一个元素的 Gradient 会通过 mean / variance 路径与同组其他元素发生耦合。

本课程当前不要求你手推完整 Jacobian，但必须建立这个直觉：

- Dropout / ReLU 更接近 elementwise operation;
- Normalization 是 group-level operation;
- 所以它的 backward 不能只看“自己这一格”的局部导数。

【科研洞察】

以后看 PyTorch Autograd，你会发现 LayerNorm / BatchNorm 的 backward 内部比 ReLU 复杂很多，这不是框架故意复杂化，而是因为 Forward 本身就把一组元素通过统计量联系到了一起。

十七、如果怀疑数值尺度失控，科研实验里怎么诊断？

不要只盯最终 Loss。可以直接记录中间层的统计量：

- activation mean;
- activation std / RMS;
- activation max absolute value;
- gradient norm;
- parameter norm;
- 是否出现 NaN / Inf。

最简单的 PyTorch 思路：

```
1 # 假设 h 是某一层 hidden state
2 print("shape =", h.shape)
3 print("mean  =", h.mean().item())
4 print("std   =", h.std(unbiased=False).item())
5 print("max|x|=", h.abs().max().item())
```

如果想系统监控很多层，可以用 forward hook，但零基础阶段先不要急着写复杂 Hook 框架。

一个实用诊断顺序

如果训练突然不稳定，可以先问：

1. 输入数据尺度是否异常？
2. Initialization 是否合理？

3. 每层 Activation std 是否逐层爆大 / 缩小?
4. Gradient norm 是否异常?
5. Learning Rate 是否过大?
6. Norm Layer 是否放错位置、axis / normalized shape 是否写错?
7. train/eval 行为是否切换正确?

【常见误区 / 风险】

不要看到 Activation Std 漂移就立即下结论“必须加某种 Norm”。尺度日志是**诊断证据**，真正修改架构还要结合模型类型和对照实验。

十八、Pre-Norm 与 Post-Norm：先建立最小直觉

Transformer 中 Normalization 不是“有没有”这么简单，**放在哪**也很重要。

一个极简 Residual Block 可以写成：

$$X \rightarrow F(X) \rightarrow X + F(X)$$

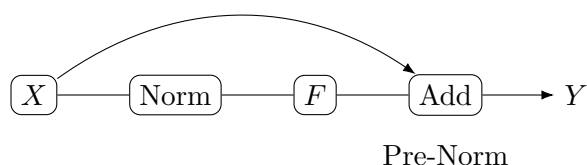
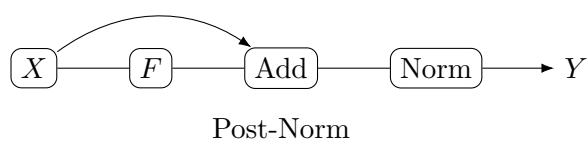
Post-Norm 的直觉形式：

$$Y = \text{Norm}(X + F(X))$$

Pre-Norm 的直觉形式：

$$Y = X + F(\text{Norm}(X))$$

二者的计算图和 Gradient Flow 不完全相同。现代 Transformer 中你会经常看到 Pre-Norm 或其变体，但本讲只要求认出这两个结构，不要求现在判断哪一个“一定更好”。



十九、Normalization 不能解决什么？

Normalization 很重要，但它不是“训练稳定万能药”。它不能自动解决：

- Learning Rate 明显过大；
- 数据标签错误；

- Loss Function 设计错误；
- Data Leakage；
- 严重 Overfitting；
- 模型架构本身不适合任务；
- Gradient 爆炸的所有来源；
- 极端数值精度问题。

【常见误区 / 风险】

看到“加了 Normalization 后训练更稳定”，只能说明观察到稳定性改善；如果论文进一步声称具体机制，还需要额外消融和机制证据。不要从一个训练曲线直接跳到唯一因果解释。

二十、做实验时应该记录哪些 Normalization 信息？

以后看论文或自己写实验，至少要明确：

1. Normalization 类型：BatchNorm / LayerNorm / RMSNorm / 其他；
2. 放置位置：Linear 前后？Activation 前后？Residual 前后？
3. normalized shape / axis；
4. ϵ ；
5. 是否有 learnable affine parameter；
6. train / eval 是否行为不同；
7. 是否有 running statistics；
8. 和初始化、LR、Optimizer 是否一起变化。

这也是以后做 Ablation 时需要控制的变量。

二十一、把本讲串回整个课程

到现在为止，训练稳定性主线已经可以串起来：

Initialization → Activation Scale → Gradient Scale → Learning Rate → Optimizer → Normalization

它们不是互相替代，而是在不同环节控制同一个深层系统中的数值传播问题。

【本节小结】

本讲最低必须真正掌握：

1. Normalization 关注的是数值分布与尺度，不是 Shape；
2. 标准化的基本形式是 $(x - \mu) / \sqrt{\sigma^2 + \epsilon}$ ；
3. 特征尺度会直接影响 Gradient 尺度；
4. 深层网络的 Activation Mean / Std 会逐层漂移；
5. 极端尺度会把某些 Activation 推进饱和区；
6. γ, β 让标准化后仍保留可学习的缩放与平移能力；
7. 最重要的问题之一是：**到底沿哪些 Axis 计算统计量？**

二十二、与后续课程的接口

下一讲：第 46 讲 BatchNorm，重点回答：

“如果统计量来自一个 Mini-Batch，会发生什么？Training 和 Eval 为什么不一样？”

第 47 讲：LayerNorm 与 RMSNorm，重点回答：

“Transformer / LLM 为什么更喜欢沿 hidden dimension 做归一化？”